

Plan MVP — Juego de cartas multijugador "viuda"

Contexto

Se está construyendo desde cero un MVP de un juego de cartas multijugador en línea. El frontend ya tiene un template limpio de Vite + React 19 (JavaScript, sin TypeScript). El backend aún no existe y se implementará en Python con FastAPI + WebSockets nativos.

Decisiones de alcance acordadas:

- **Reglas del juego:** propias e inventadas, se definirán en un bloque posterior (no bloquean la infraestructura).
- **Baraja:** francesa de 52 cartas + 2 jokers = **54 cartas**.
- **Jugadores:** 2 a 9 jugadores en una **única sala fija** (sin sistema de salas múltiples). **Espectadores:** quien entre cuando ya hay 9 jugadores en la sala de espera, o cuando la partida ya está en curso, queda como **espectador** (ve el estado público pero no juega).
- **Lobby:** pantalla de entrada pide nickname → muestra opción "Unirme a la sala de espera" + lista en vivo de quienes ya esperan → cuando hay ≥ 2 esperando, **cualquier jugador de la sala de espera puede iniciar la partida**.
- **Persistencia: sin base de datos, sin autenticación** — todo el estado vive en memoria del proceso del servidor. Los jugadores se identifican por un nickname temporal.
- **Granularidad:** bloques pequeños y verificables — cada bloque debe terminar con algo testeable manualmente.
- **Estilo de trabajo:** avanzar bloque a bloque; no se intentará hacer todo de golpe.

Estado actual del repo:

- `package.json` — React 19.2 + Vite 8 + ESLint, scripts `dev/build/lint/preview`.
- `src/` — esqueleto generado por `create-vite` (App.jsx, main.jsx, index.css, App.css, assets).
- `index.html`, `vite.config.js`, `eslint.config.js`.
- **No hay backend ni lógica del juego.**

Arquitectura objetivo

```
viuda/
├── frontend/           # actual contenido de viuda/ movido aquí
│   ├── src/
│   ├── package.json
│   └── vite.config.js
└── backend/           # nuevo
    ├── app/
    │   ├── main.py    # FastAPI app + endpoint /ws
    │   ├── room.py    # GameRoom: estado en memoria, jugadores
    │   ├── protocol.py # esquemas de mensajes cliente↔servidor
    │   ├── deck.py    # baraja francesa + 2 jokers, barajar, repartir
    │   └── game.py    # (bloque 7) máquina de estados de la partida
```

```
| requirements.txt  
| README.md
```

Flujo de comunicación: un único endpoint WebSocket `/ws`. Mensajes JSON con campo `type` discriminante. El servidor mantiene una `GameRoom` global (singleton en memoria) con la lista de jugadores conectados y el estado de la partida en curso.

Bloques del MVP

Cada bloque tiene **Objetivo**, **Archivos clave** y **Cómo verificar**. Avanzamos uno a uno; al terminar cada bloque, se valida antes de pasar al siguiente.

Bloque 0 — Reorganización del repositorio

Objetivo: separar `frontend/` y crear esqueleto `backend/`.

- Mover el contenido actual de la raíz a `frontend/` (todo lo que es de Vite/React).
- Crear `backend/app/`, `backend/requirements.txt` (`fastapi`, `uvicorn[standard]`, `websockets`), `backend/README.md`.
- Añadir un `README.md` raíz con instrucciones de arranque de ambos lados.
- (Opcional) `.gitignore` raíz combinando Node y Python.

Verificar: `cd frontend && npm run dev` sigue funcionando; la estructura `backend/` existe vacía pero con `requirements.txt` válido.

Bloque 1 — Backend FastAPI mínimo con endpoint WebSocket

Objetivo: servidor FastAPI con un endpoint `/ws` que acepte conexiones y haga echo de los mensajes recibidos.

- `backend/app/main.py`: `FastAPI()`, `@app.websocket("/ws")` que acepta, recibe texto y responde echo.
- CORS abierto para `http://localhost:5173` (origen de Vite por defecto).
- Script de arranque: `uvicorn app.main:app --reload --port 8000`.

Verificar: desde un cliente WS sencillo (extensión de navegador, `wscat`, o un pequeño script Python) conectar a `ws://localhost:8000/ws`, enviar `"hola"`, recibir `"hola"`.

Bloque 2 — Sala única en memoria + roles (jugador / espectador / en espera)

Objetivo: modelo `GameRoom` con tres conjuntos de conexiones: **en espera** (quieren jugar la próxima partida, máx 9), **jugadores activos** (de la partida en curso) y **espectadores**. Broadcast del roster en vivo.

- `backend/app/room.py`: clase `GameRoom` con `connections: dict[connection_id, Connection]`, donde cada conexión tiene `nickname` y `role ∈ {waiting, playing, spectator}`. Estado de la sala: `idle` (sin partida) o `in_game`.

- Métodos: `connect(nickname)`, `join_waiting_list(conn)`, `disconnect(conn)`, `broadcast(message)`, `start_game(initiator)`.
- En `/ws`: al conectar, esperar primer mensaje con el `nickname`; rechazar **solo** si el nickname está en uso. Tras aceptar, el rol inicial es:
 - `waiting` si la sala está `idle` y hay menos de 9 esperando.
 - `spectator` si la sala está `in_game`, o si ya hay 9 esperando.
- Al recibir `join_waiting_list` desde un espectador, promoverlo a `waiting` si hay cupo y la sala está `idle`.
- Al desconectar, limpiar y broadcast.
- Mensaje `room_state` incluye las tres listas (`waiting`, `playing`, `spectators`) y la fase (`idle/in_game`).

Verificar: abrir 11 clientes con nicknames distintos; los primeros 9 quedan en `waiting`, el 10º y 11º como `spectator`. Nickname duplicado → rechazo. Si uno de los 9 se desconecta, queda cupo y el siguiente `join_waiting_list` de un espectador funciona. Si la sala está `in_game`, cualquier nuevo entrante es espectador hasta que termine.

Bloque 3 — Frontend: pantalla de nick + sala de espera + iniciar partida

Objetivo: UI con dos pantallas — entrada de nick y lobby de espera — sincronizadas vía WS.

- `frontend/src/App.jsx`: orquesta el flujo según el estado (`nick_form` → `lobby` → `in_game`).
- Pantalla 1 — **Entrada de nick**: input + botón "Continuar". Al enviar, abre el WebSocket y manda el `nickname`. Si hay error (nick duplicado) se muestra en línea.
- Pantalla 2 — **Lobby**: muestra tres listas (en espera / espectadores / jugando si ya hay partida). Botón **"Unirme a la sala de espera"** visible solo si el usuario es espectador y queda cupo y la sala está `idle`. Botón **"Iniciar partida"** visible solo si el usuario está en `waiting` y hay ≥ 2 en espera; al pulsarlo envía `start_game`.
- Nuevo hook `frontend/src/hooks/useGameSocket.js`: encapsula `new WebSocket('ws://localhost:8000/ws')`, manejo de `onopen/onmessage/onclose`, envío del nickname inicial, exposición del estado de la sala y de funciones `joinWaitingList()` / `startGame()`.
- Componente `Lobby.jsx` con las tres listas y los botones contextuales.

Verificar: abrir varias pestañas, entrar con nicks distintos; las tres listas se sincronizan en vivo. Un espectador puede pulsar "Unirme a la sala de espera" y aparece en `waiting`. Un jugador en `waiting` pulsa "Iniciar partida" → la sala pasa a `in_game` y los espectadores ven el cambio. Nick duplicado → error en pantalla de entrada.

Bloque 4 — Protocolo de mensajes tipado y enrutador genérico

Objetivo: formalizar el contrato cliente↔servidor antes de añadir lógica de juego.

- `backend/app/protocol.py`: modelos Pydantic para cada `type` de mensaje (entrante: `join`, `ping`, `chat`, futuro `play_card`; saliente: `room_state`, `pong`, `error`, futuro `game_state`).
- Enrutador en `main.py` que despacha por `type` a un handler.
- En frontend, función `send(type, payload)` y `onMessage(type, handler)` registradas en el hook.
- Añadir `ping/pong` como prueba del enrutador y un mini chat opcional para validar el flujo bidireccional.

Verificar: ping desde el frontend → pong de vuelta. Mensaje con **type** desconocido → respuesta **error** clara. Si se añade chat, los mensajes se ven en todas las pestañas.

Bloque 5 — Modelo de baraja y reparto (sin reglas aún)

Objetivo: representar la baraja francesa + 2 jokers, barajar y repartir.

- `backend/app/deck.py`: `Card(suit, rank)` con `suit` ∈ {♠♥♦♣, joker} y `rank` apropiado; `build_deck()` devuelve 54 cartas; `shuffle()`; `deal(n_players, cards_per_player)`.
- Acción del servidor `start_game` (provisional, sin reglas): congela la lista `waiting` como jugadores activos (rol `playing`), reparte cartas, envía a cada jugador **solo su mano** (mensaje privado) y a todos (incluidos espectadores) el conteo de cartas restantes / posición de los demás. La sala pasa a `in_game`.
- Distinción explícita entre estado público (broadcast a todos: jugadores + espectadores) y estado privado (per-socket, solo manos de jugadores activos).
- Cualquier entrante mientras `in_game` es espectador y recibe solo el estado público.

Verificar: desde un cliente, disparar `start_game`. Cada pestaña recibe únicamente su propia mano; ninguna ve las cartas de las otras. El total de cartas repartidas + restantes = 54.

Bloque 6 — UI de mesa: render de cartas y posiciones

Objetivo: visualizar la mesa, la mano propia y los otros jugadores (sin interactividad de jugada todavía).

- Componentes en `frontend/src/components/`: `Table.jsx`, `Hand.jsx`, `Card.jsx`, `OpponentSeat.jsx`.
- Mano propia: cartas visibles. Otros jugadores: dorso de cartas con el conteo.
- Posicionamiento básico alrededor de una "mesa" circular (CSS flex/grid o transforms simples).
- Sin librerías de animación todavía — solo render estático correcto.

Verificar: con 3-4 pestañas, cada una ve su mano y a los otros jugadores como asientos con dorsos. Refrescar tras un nuevo `start_game` actualiza correctamente.

Bloque 7 — Definición e implementación de las reglas del juego

Objetivo: el usuario aporta las reglas; se diseña la máquina de estados.

- **Antes de codificar**, sesión de preguntas para capturar: turno (quién empieza, orden), jugadas válidas, función de la "viuda" (descarte/mano oculta), condiciones de victoria, scoring, número de rondas.
- `backend/app/game.py`: clase `Game` con estado (`turn`, `phase`, `widow`, `discard_pile`, `scores`, ...) y métodos `is_valid_move(player, move)`, `apply_move(player, move)`, `is_finished()`.
- Tests unitarios mínimos para `is_valid_move` y transiciones clave.

Verificar: tests unitarios pasan; el estado se serializa a JSON correctamente y los mensajes `game_state` reflejan lo esperado.

Bloque 8 — Interactividad: jugar cartas con validación servidor

Objetivo: cerrar el bucle de juego — clic en una carta → servidor valida → broadcast del nuevo estado.

- Frontend: handler `onClick` en `Card.jsx` que envía `play_card` con el id de la carta.
- Backend: validar con `Game.is_valid_move`; si es válido, aplicar y broadcast de `game_state` (versión pública) + mensajes privados a cada jugador con su mano actualizada.
- En caso de jugada inválida, mensaje `error` solo al jugador que la intentó.
- Indicador visual de "turno actual" y deshabilitar interacción cuando no toca.

Verificar: partida completa de principio a fin entre 2-3 pestañas. Intentos de jugar fuera de turno o con cartas que no se tienen → rechazo limpio.

Bloque 9 — Pulido MVP

Objetivo: robustez mínima para uso real.

- Manejo de desconexión durante partida: pausar o expulsar (decisión a tomar con el usuario en su momento).
- Fin de partida: la sala vuelve a `idle`, los `playing` regresan a `waiting`, los espectadores que pulsen "Unirme" pueden pasar a `waiting` para la siguiente.
- Reinicio de partida (`new_game`) sin reiniciar el servidor — disparable por cualquier jugador en `waiting`.
- Mensajes de error legibles en UI.
- README raíz con instrucciones de arranque y screenshots.

Verificar: desconectar un jugador a mitad de partida y reconectarlo (o reemplazarlo según se decida); reiniciar partida sin tirar el servidor; experiencia fluida con 4-5 jugadores reales.

Archivos críticos a crear/modificar

Backend (nuevo):

- `backend/app/main.py` — FastAPI app y endpoint `/ws` (Bloques 1, 4).
- `backend/app/room.py` — `GameRoom` en memoria (Bloque 2).
- `backend/app/protocol.py` — esquemas Pydantic (Bloque 4).
- `backend/app/deck.py` — baraja 54 cartas (Bloque 5).
- `backend/app/game.py` — reglas y máquina de estados (Bloque 7).
- `backend/requirements.txt` — dependencias Python (Bloque 0).

Frontend (modificar/crear):

- `frontend/src/App.jsx` — orquestación de pantallas (Bloque 3).
- `frontend/src/hooks/useGameSocket.js` — cliente WS (Bloques 3, 4).
- `frontend/src/components/Lobby.jsx` — pantalla de espera (Bloque 3).
- `frontend/src/components/Table.jsx`, `Hand.jsx`, `Card.jsx`, `OpponentSeat.jsx` — UI de juego (Bloques 6, 8).

Verificación end-to-end del MVP

Tras el Bloque 9, debe ser posible:

1. Arrancar backend con `uvicorn app.main:app --reload --port 8000` desde `backend/`.
2. Arrancar frontend con `npm run dev` desde `frontend/`.

3. Abrir 2-9 pestañas/dispositivos, entrar con nicknames distintos.
4. Iniciar partida, jugar de principio a fin respetando las reglas definidas en el Bloque 7.
5. Ver el resultado final y poder iniciar una nueva partida sin reiniciar el servidor.

Siguiente paso inmediato

Empezar por el **Bloque 0** (reorganizar **frontend/** y crear esqueleto **backend/**). Es un cambio puramente estructural, sin lógica nueva, y deja el terreno listo para el Bloque 1.